

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Pseudocode Writing Task (Algorithm Design)

COURSE NAME: COMPUTER VISION

COURSE CODE: ITA0515

Slot B

COURSE OUTCOME COVERED

CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)

Assessment Tool Weightage: 20%

QUESTIONS:5

Total Marks 20

Duration :60 Minutes Annexure A
Pseudocode Writing Task (Algorithm Design)

Code of Conduct:

I K.Mounith(192425090) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K.Mounith

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation	
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach	
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs	
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear	

Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient	
--------------	-----	--	--	-----------------------------	------------------------------	--

Pseudocode Writing Tasks: Design a clear and structured pseudocode for the given problem by organizing the solution into logical stages of a computer vision pipeline. Ensure proper use of control flow, modular steps, and justification of key operations to satisfy the given constraints.

1.Real-Time Object Detection Pipeline A

system must:

- A. Capture video frames
 - B. Detect objects using a trained model
 - C. Draw bounding boxes and labels
 - D. Display results in real time
- Constraints:
- Must process frames efficiently
 - Handle multiple objects

Write detailed pseudocode with modular steps for detection, visualization, and efficient looping.

A. START

1. LOAD required libraries

2. LOAD the trained object detection model

3. OPEN video camera

4. WHILE camera is running:

- a. CAPTURE one video frame
- b. IF frame is not available:
 BREAK the loop
- c. RESIZE frame if required for faster processing
- d. SEND frame to the trained detection model
- e. GET detected objects:
 - Object class
 - Confidence
 - Bounding box coordinates
- f. FOR each detected object:
 - IF confidence $\geq$ threshold:
 - DRAW bounding box
 - DRAW object label

DRAW confidence score

g. DISPLAY the processed frame

h. IF user presses 'q':
 BREAK the loop

5. RELEASE camera

6. CLOSE all OpenCV windows

END

2. Image Difference Detection System A
system must:

- A. Read two images
- B. Compare them
- C. Highlight differences Constraints:
 - Handle noise
 - Ensure accurate difference detection

Write pseudocode including preprocessing, comparison logic, and visualization.

A. START

1. IMPORT OpenCV and NumPy

2. READ Image 1 and Image 2

3. CHECK whether both images are loaded
 IF either image is not loaded:
 DISPLAY error
 STOP

4. PREPROCESS images

- a. Resize both images to the same size
- b. Convert both images to grayscale
- c. Apply Gaussian Blur to reduce noise

5. COMPARE the two images

- a. Calculate absolute difference between images
- b. Apply threshold to identify significant differences
- c. Apply morphological operation to remove small noise

6. FIND DIFFERENT REGIONS

- a. Find contours in the difference image
- b. FOR each contour:

IF contour area > minimum threshold:
Find bounding box
Store the difference region

7. VISUALIZE DIFFERENCES

- a. Draw bounding boxes around different regions
- b. Highlight the differences on Image 2
- c. Display original images and difference image

8. DISPLAY results

9. WAIT for user input

END

3. Video Frame Extraction System A
system must:

- A. Read video input
- B. Extract frames at fixed intervals
- C. Save selected frames

Constraints:

- Efficient storage
- Avoid redundant frames

Write structured pseudocode for frame selection and saving logic.

A. START

1. IMPORT OpenCV and OS

2. OPEN the video file

3. CHECK whether video is opened

IF video cannot be opened:

DISPLAY error

STOP

4. GET video information

- a. Get FPS
- b. Get total number of frames

5. SET FRAME_INTERVAL

Example: Extract one frame every 5 seconds

6. CREATE output folder

7. INITIALIZE frame_count = 0

INITIALIZE saved_count = 0

8. WHILE video has frames:

a. READ next frame

b. IF frame is not available:
 BREAK

c. Calculate current time:
 time = frame_count / FPS

d. IF current time reaches FRAME_INTERVAL:

 i. Check whether frame is different
 from the previously saved frame

 ii. IF sufficiently different:
 SAVE frame
 Increment saved_count

 iii. Update next extraction time

e. Increment frame_count

9. RELEASE video

10. DISPLAY total number of saved frames

END

4. Background Subtraction Pipeline A
surveillance system must:

- A. Read video stream
- B. Separate foreground from background
- C. Highlight moving objects Constraints:
 - Handle dynamic background
 - Reduce noise

Write pseudocode including background modeling, subtraction, filtering, and display.

START

1. Import OpenCV

2. Open the video stream

```
3. Check whether the video is opened
   IF not opened:
       Display error
       STOP

4. Create an adaptive background model using MOG2

5. WHILE video is running:

    a. Read the current frame

    b. IF frame is not available:
        BREAK

    c. Apply background subtraction
        foreground = current frame - background

    d. Apply noise reduction
        - Gaussian Blur
        - Morphological Opening

    e. Apply thresholding to obtain a clean foreground mask

    f. Find contours of foreground objects

    g. FOR each contour:
        IF contour area > minimum area:
            Find bounding box
            Draw bounding box around moving object

    h. Display:
        - Original video
        - Foreground mask
        - Highlighted moving objects

    i. IF user presses 'q':
        BREAK

6. Release video

7. Close all windows

END
```

5. Edge Detection Pipeline for Real-Time Video A system must:

- A. Capture video frames
- B. Apply edge detection
- C. Display results

Constraints:

- Real-time processing
- Maintain clarity of edges

Write pseudocode with preprocessing, edge detection, and visualization modules.

A. START

1. Import OpenCV

2. Open the video camera

3. Check whether the camera is available

IF camera is not available:

Display error

STOP

4. WHILE video is running:

a. Capture a video frame

b. Resize frame for faster processing

c. Convert frame to grayscale

d. Apply Gaussian Blur to reduce noise

e. Apply Canny Edge Detection

f. Display:

- Original frame

- Edge-detected frame

g. IF user presses 'q':

BREAK

5. Release camera

6. Close all windows

END